$ mysql -e "SHOW CREATE TABLE orders" --normalize

資料庫正規化

// 1NF → 2NF → 3NF 的拆表之旅

1NF
原子性

2NF
消除部分相依

3NF
消除遞移相依

反正規化
OLAP 的選擇

$ cat schema-plan.md

今日路線

01 / 病灶
資料冗餘與三種異常

02 / 三步拆表
1NF 原子性 → 2NF → 3NF

03 / 權衡
JOIN 的代價

04 / 反正規化
OLTP vs OLAP 的選擇

目標：每項事實只存一份 — 改一次，全站生效。

PART 01

病灶：冗餘與異常

同一件事實，存了 N 份

資料冗餘：未正規化的訂單表

order_id	customer_name	customer_email	product_name	price
1	張三	zhang@example.com	筆記型電腦	30000
2	張三	zhang@example.com	滑鼠	500
3	李四	li@example.com	鍵盤	1200

浪費空間
張三的姓名和 email 重複出現

更新惡夢
張三改 email，要 UPDATE 多筆記錄

三種資料異常

插入異常
想新增「資料庫設計」課程，但還沒有學生選修 → 存不進去

更新異常
王老師改名，要改所有課程記錄，漏一筆就不一致

刪除異常
刪掉最後一個選課學生，課程資訊跟著消失

ROOT CAUSE

三種異常同一個病根：不相干的事實擠在同一張表裡。

CHECKPOINT_01

「王老師」改名為「王教授」，要 UPDATE 37 筆課程記錄，漏了 3 筆。這是哪種異常？

A 插入異常：沒有學生就存不了課程

B 更新異常：改一筆漏一筆，資料不一致

C 刪除異常：刪掉學生，課程也跟著消失

PART 02

三步拆表

1NF → 2NF → 3NF，一步一個腳印

1NF：每格只放一個值

BEFORE — 違反原子性

student	courses
張三	資料庫, 演算法, 網路

AFTER — 1NF

student	course
張三	資料庫
張三	演算法
張三	網路

消除重複群組

欄位原子性

建立主鍵

2NF：消除部分相依

主鍵是 (order_id, product_id)，但 product_name 只依賴 product_id —— 這就是部分相依

BEFORE — 部分相依

order_id	product_id	product_name	qty
1	P1	筆電	1
2	P1	筆電	3

AFTER — 拆成兩張表

order_id	product_id	qty
1	P1	1

product_id	product_name
P1	筆電

非主鍵欄位必須依賴整個主鍵，不能只依賴主鍵的一部分。

3NF：消除遞移相依

student_id → dept_id → dept_name：dept_name 透過 dept_id 間接依賴主鍵 —— 這就是遞移相依

BEFORE — 遞移相依

student_id	name	dept_id	dept_name
S1	張三	D1	資工系
S2	李四	D1	資工系

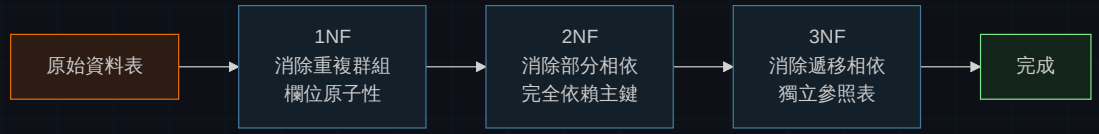
AFTER — 獨立參照表

student_id	name	dept_id
S1	張三	D1

dept_id	dept_name
D1	資工系

非主鍵欄位之間不該互相依賴 — 系名改一次，全系學生都對。

正規化流程一覽



注意：正規化並非越高階越好。一般系統達到 **3NF 已足夠**， 過度正規化只會讓查詢複雜度爆炸。

PART 03

權衡與反正規化

沒有免費的午餐：JOIN 是代價

正規化給你的

每項事實只存一份
節省空間、改一次全站生效

資料完整性
三種異常從根源消失

擴展性
結構清晰，容易適應變更

正規化要你付的

JOIN 變多
撈完整資訊要跨多張表

查詢變慢
複雜 JOIN 影響讀取效能

理解成本
高度正規化的結構較難一眼看懂

OLTP vs OLAP：選邊站

比較項目	正規化	反正規化
資料冗餘	幾乎無	有，重複儲存
更新效能	快，只改一處	慢，要改多處
查詢速度	可能較慢（JOIN）	快，少 JOIN
一致性維護	容易	需要同步機制
適用場景	**OLTP** 交易系統	**OLAP** 報表／資料倉儲

STRATEGY

從正規化開始，根據效能測試結果再選擇性反正規化 — 讀取密集、報表分析、星型架構才是反正規化的主場。

今日重點

病根
不相干的事實擠同一張表 → 三種異常

三步拆表
1NF 原子性 → 2NF 部分相依 → 3NF 遞移相依

3NF 就夠
過度正規化讓查詢複雜度爆炸

先正規化再權衡
OLTP 正規化；OLAP 可適度反正規化

正規化的本質：每項事實，只存一份。

Schema normalized.

下一步：資料庫索引，讓 JOIN 不再是藉口

```
$ mysql -e "EXPLAIN SELECT ..."
1NF → 2NF → 3NF → 需要時再反正規化
```